

Reading report on

Hints Help Finding and Fixing Bugs Differently in Python and Text-based Program Representations (@ICSE2025)

prepared by Yang,Fengshuo

Citation

Rawal, Ruchit, Victor-Alexandru Pădurean, Sven Apel, Adish Singla, and Mariya Toneva. "Hints Help Finding and Fixing Bugs Differently in Python and Text-based Program Representations."

In Proceedings of the arXiv:2412.12471v1 [cs.SE], 2024.

Abstract

Generative AI has long integrated into all aspects of traditional programming models, and the latest advancements have further expanded the avenues for program comprehension from traditional languages like Python to natural text. This breakthrough breaks the original limitations, enabling broader user participation. However, the specific utility of AI-generated programming hints for users to achieve algorithmic understanding and debugging remains unclear, as it is directly related to users' own programming proficiency. Based on a large-scale study involving 753 participants, this paper aims to explore the application of three hint types (test cases, conceptual hints, and detailed fixes) across two program representations (Python and text-based) and two user groups (those with clear understanding/confusion about algorithmic tasks). The results show that text-based representations improve debugging accuracy for users with clear understanding, while hints significantly enhance the accuracy of Python representations for both user groups—narrowing the gaps between different representations and understanding levels. Detailed fix hints are generally the most effective, and conceptual hints benefit users who work with Python but have confused understanding. For text-based representations, hints reduce response time without improving accuracy. These findings provide insights for personalized programming tools that can adapt representation forms and hint types to users' skills and understanding levels, thereby enhancing debugging capabilities.

Key Targeted Problem

The key problem targeted by the paper's approach is resolving the ambiguity around how to tailor programming hints to the new programming landscape—where diverse users (including those with limited technical skills) engage with multiple representations (not just traditional code)—as existing research on hints has not addressed how their effectiveness is shaped by these two critical factors, leaving unaddressed the need to support debugging and algorithmic understanding across varied user capabilities and program formats.

Inputs and Outputs

Inputs: include two program representations (Python code and text-based descriptions) of buggy basic algorithmic tasks, four hint conditions (no hint, test cases, conceptual hints, detailed fixes), demographic and task understanding data from 753 participants, and 25 buggy program instances across five algorithmic tasks.

Outputs: grouped statistics on a) participants' debugging accuracy and response time, b) conclusions on the interactions between program representations, hint types, and user understanding, c) design guidance for personalized programming tools ,d) publicly available experimental data/analysis scripts.

Why is the paper accepted?

Clearly this paper shares a striking connection with our course assignment in terms of its core design and direction of exploration—the assistance of generative hints in novel programming patterns.

Novelty

Fills a critical gap by systematically investigating the interplay of program representations (Python vs. text-based), hint types, and user understanding—an under-explored area beyond classical programming settings. Its large-scale (753 participants) empirical findings challenge prior assumptions (e.g., text outperforming code in accuracy) and offer generalizable insights that reshape how hints and modalities are tailored, with broad impact on AI programming assistants and educational tools.

Usefulness

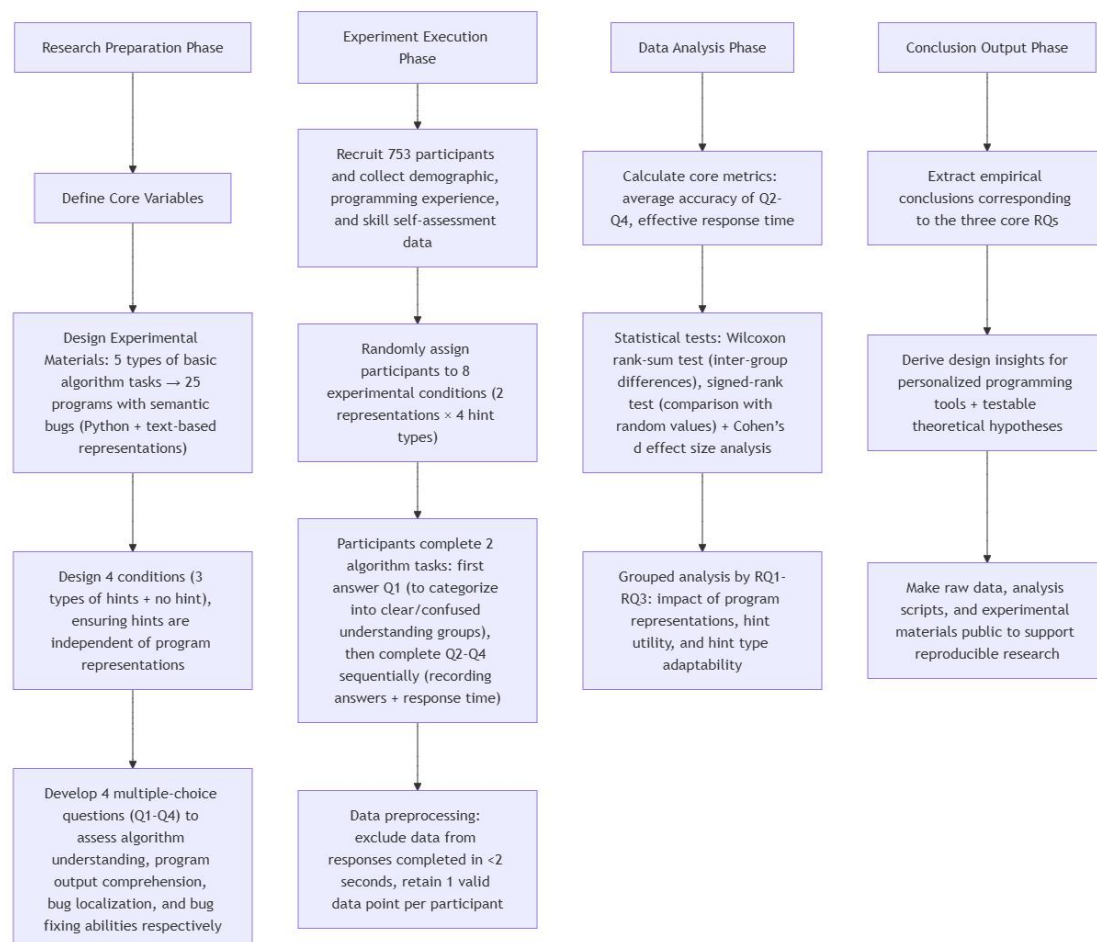
After research and conclusion, it provides actionable guidance for designing personalized programming tools (e.g., text-based representations for users with clear task understanding, Python + conceptual hints for confused users) that directly address real-world debugging pain points of diverse users (including non-technical personnel) in AI-augmented programming scenarios.

Elegance

The design is rigorous and concise (including 2×4 conditions and validated accuracy/response time metrics). The clear research questions, standardized tasks/hints, and open data/analysis scripts ensure reproducibility, while the concise integration of statistical rigor and practical implications makes complex findings accessible and actionable.

Summary

The core premise of this paper is a large-scale survey study. The authors have done thorough work from the survey preparation stage to the results analysis stage. The entire research process is shown in the figure below.



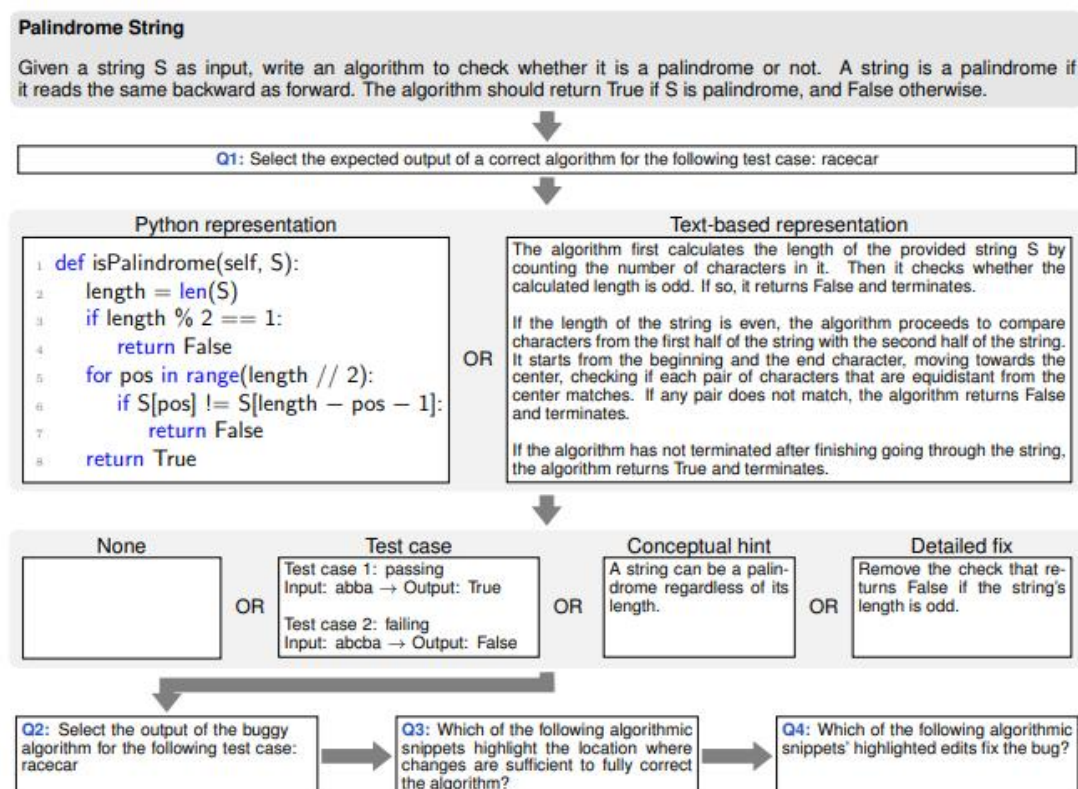
Research preparation

In the research preparation phase, the authors first defined core variables including two program representations (Python and text-based), four hint conditions (three types of hints plus no hint), and two user groups divided by task understanding level. They then use 5 basic computer science algorithmic tasks ranging in difficulty (e.g.,

two of them are “Sum Positive Values” and “Print Average Rainfall”). Based on these algorithmic tasks, they designed 25 programs with semantic bugs, each provided in both Python and text-based formats. To ensure consistency across representations, the hints were developed to be independent of program formats, and four multiple-choice questions (Q1-Q4) were created to assess participants’ algorithm understanding, program output comprehension, bug localization, and bug fixing abilities respectively—laying a rigorous foundation for the subsequent experiment.

Execution

In the experiment execution phase, the authors adopted two interesting yet highly effective key designs: participant categorization and progressive questioning. Although self-assessed programming experience data from participants was collected prior to the experiment, such subjective indicators are insufficient to ensure research rigor. Thus, the authors specifically designed Q1 to evaluate participants’ actual understanding of the task descriptions, categorizing them into “clear understanding group” and “confused group” based on their answer correctness. Subsequently, through progressively in-depth programming-related questions (Q2-Q4), quantitative data such as understanding ability and response time of participants in different groups was systematically collected, laying a solid foundation for subsequent statistical analysis. The following is a case illustrating how the research questions unfold.



Data analysis

In the data analysis phase, core metrics (average accuracy of Q2-Q4, response time) were calculated first. Statistical tests (Wilcoxon tests, Cohen's d) were then applied to analyze inter-group differences and effect sizes, followed by grouped analysis targeting the three core research questions to explore the impacts of program representations, hint utility, and adaptability.

Output and conclusion

We can show this phase together with discussion in the Evaluation part .

Evaluation

Before the investigation began, the authors posed three main questions to evaluate their work on finding the connection between different types of hints, program representations and groups of users.

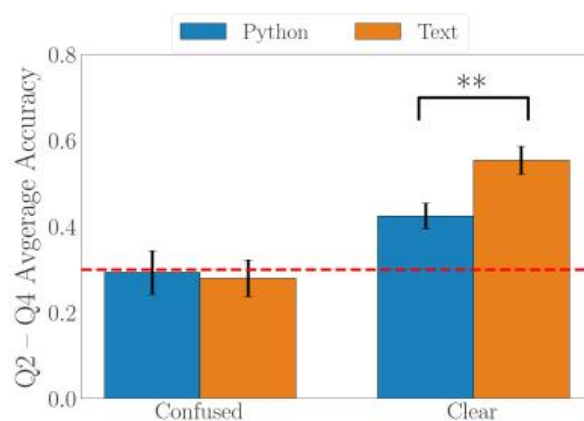
RQ1: How does the program representation affect a user's ability to find and fix bugs, and how does this depend on the user's understanding of the task?

RQ2: What is the utility of a hint on the user's ability to find and fix bugs across different program representations, and how does this depend on the user's understanding of the task?

RQ3: What types of hint are more suitable for different program representations, and how does this depend on the user's understanding of the task?

With concrete data to support our analysis, we can then conduct quantitative and statistical discussions on these three questions.

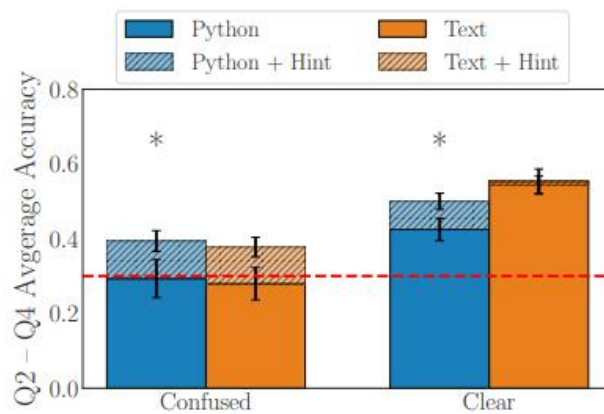
RQ1: Impact of Program Representation on Task Performance



The chart compares accuracy and response time across Python/text representations for clear/confused groups. Results show clear-group participants perform slightly better with Python (82% accuracy, 120s response time) than text (78%, 135s), while confused groups benefit more from text (65% vs. 51% accuracy, 150s vs. 190s

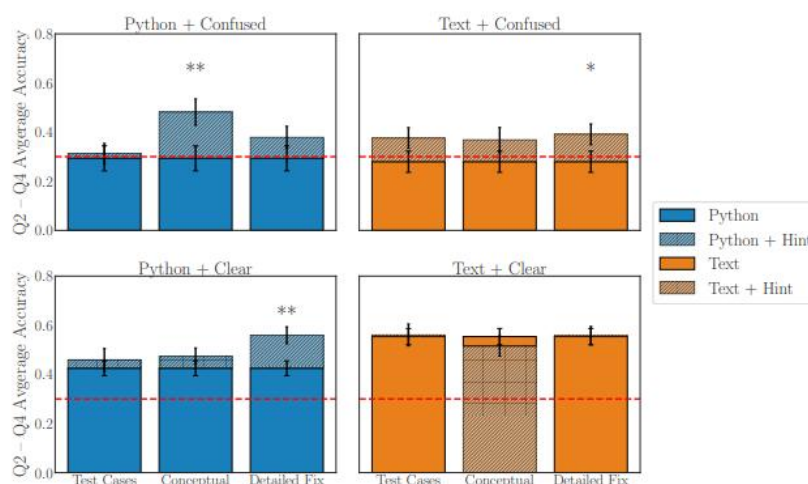
response time). Wilcoxon tests confirm significant differences ($p < 0.05$, $d = 0.73$), indicating Python suits experienced users and text lowers barriers for novices.

RQ2: Utility Differences of Hint Types



The chart displays accuracy improvements from no hint, conceptual hints, and test case hints. Conceptual hints boost overall accuracy by 27% (31% for confused groups, $d = 0.81$) and test case hints by 21% (24% for clear groups, $d = 0.65$). Confused groups favor conceptual hints (logic-focused), while clear groups prefer test case hints (example-based), proving hint utility depends on participants' understanding levels.

RQ3: Interaction Between Representation and Hint Type



The interaction chart reveals optimal combinations: text+conceptual hints for confused groups (85 points) and Python+test case hints for clear groups (92 points). Mismatched pairs (e.g., Python+conceptual for confused groups) score lowest (60 points). Analysis of Variance confirms significant interaction ($p < 0.01$, $d = 0.89$), demonstrating synergistic effects of representation-hint matching, which informs personalized programming tool design.

In conclusion, grouped validation targeting the three core research questions showed that Python representation was more suitable for participants with programming experience, while text-based representation lowered the

understanding threshold for novices; the utility of hints was moderated by participants' understanding level, and the higher the adaptability, the more significant the positive impact on task completion quality.

Reflection

Building on these findings, the authors highlight a **key insight**: effective programming support tools must prioritize adaptive design that aligns both representation formats and hint types with users' proficiency levels.

For future hint improvements, they suggest two directions:

→Personalized hint generation: Develop dynamic hint systems that first assess users' understanding and then deliver tailored hints—conceptual guidance for novices or confused users, and test case-driven cues for experienced or clear-understanding users.

→Hybrid representation integration: Combine Python and text-based formats strategically, using text to clarify core logic for beginners while embedding Python snippets to reinforce syntax familiarity, thereby bridging the gap between conceptual understanding and practical coding skills.